



Exotic Options: Pricing a Rainbow Option of Two Assets via Monte Carlo and Variance Reduction Techniques

Computational and Quantitative Finance with C++

Agianogloy Nikolaos-Vasileios:MFT2501

DelRosso Vasileios:MFT2509

Lathiotakis Efthymios:MFT2504

Pattas Panagiotis:MFT2510

Instructor : Englezos Nikolaos —University of Piraeus

- Exotic Options — Rainbow Options. They derive their value from multiple underlying assets (e.g two) and their payoff can have different forms. Their theoretical foundation was provided by Margrabe (1978) and later on by Stulz (1982).
- Due to the multi-asset exposure under a single derivative contract they provide a more natural risk diversification, enhanced returns and a better cost efficiency.
- Forms of Rainbow Options are :
 - Best of assets or cash : $\max(S_1, S_2, \dots, S_n, K)$
 - **Call on max/min** : $\max(\max/\min(S_1, S_2, \dots, S_n) - K, 0.0)$
 - Put on max/min : $\max(K - \max/\min(S_1, S_2, \dots, S_n), 0.0)$
 - Put 2 and Call 1 : $\max(S_1 - S_2, 0.0)$

Those options types *follow a Geometric Brownian Motion under the risk-neutral measure*:

- $dS_i(t) = rS_i(t)dt + \sigma_i S_i(t)dZ_i(t)$, $i = 1, 2$ where r represents the risk-free rate, σ_i is the volatility per asset, $Z_i(t)$ are independent Standard Brownian Motions.
- It is given that the correlation between the two assets
 $Corr(S_1, S_2) \leftrightarrow Corr(dZ_1(t), dZ_2(t)) = 0$
- Also there are no dividends on either asset : $q_1 = q_2 = 0$

Theoretical Pricing

Below is provided a quick summary of the solution on the Stochastic Differential Equation (SDE): $dS_i(t) = rS_i(t)dt + \sigma_i S_i(t)dZ_i(t)$, $i = 1, 2$

- To solve the above equation we apply Ito's Lemma :

$dG = (\frac{\partial G}{\partial t} + \frac{\partial G}{\partial S}\mu S + \frac{1}{2}\frac{\partial^2 G}{\partial S^2}\sigma^2 S^2)dt + \frac{\partial G}{\partial S}\sigma SdZ_t$ and by setting $G = \ln(S_i)$ we obtain :

$dG = (r - \frac{1}{2}\sigma^2)dt + \sigma dZ_t$ is a Generalized Wiener Process because now drift and variance rates are constants

- Next step is to integrate $\int$ on both sides for our time interval of $t \in [0, T]$ thus we get: $\ln S_{i,T} = \ln S_{i,0} + (r - \frac{1}{2}\sigma^2)T + \sigma Z_{i,T}$ or equivalently:

$S_{i,T} = S_{i,0} \exp((r - \frac{1}{2}\sigma_i^2)T + \sigma_i Z_{i,T})$ with $Z_i = \epsilon_i \sqrt{\Delta t}$ with $\epsilon_i \sim N(0, 1)$

So there is the conclusion that: $\ln S_{i,T} \sim N[\ln S_{i,0} + (r - \frac{1}{2}\sigma_i^2)T, \sigma_i \sqrt{T}]$ meaning there is a Lognormal distribution followed by each asset's price

Theoretical Pricing — cont'd

Since the existence of two assets simultaneously under the payoff's structure, with each one having its own mean and volatility, there is the need to define their prices in a **Joint Distribution** which is the key function to express possible dependencies overall. That's because here : $E[f(S_1, S_2)] \neq f(E[S_1], E[S_2])$

First, let's assume that :

- A joint vector of the prices is $(\ln S_1(T), \ln S_2(T)) = (Y_1, Y_2) \sim N(\mu, \bar{\Sigma})$, where now:

- $\mu = \begin{pmatrix} \ln S_1(0) + (r - 1/2\sigma_1^2)T \\ \ln S_2(0) + (r - 1/2\sigma_2^2)T \end{pmatrix}$

- $\Sigma = \begin{pmatrix} \sigma_1^2 T & 0 \\ 0 & \sigma_2^2 T \end{pmatrix}$ So since the Σ is diagonal, then Y_1 and Y_2 are independent normal variables.

Theoretical Pricing — Disclaimer

Since we have created a backbone for our theoretical pricing we must report here the following statement:

- By evaluating the Marginal Densities of the Prices per asset we are able to compute the exact closed formula of call-on-min payoff. Thus it is always better to directly calculate it from that formula rather than proceeding with the numerical approximations of Monte Carlo.
- In more detail the Closed-Form Formula was computed by Stulz and is:
$$C_{min}(0) = S_{1,0}e^{-q_1 T} M(d_1, -d; -\rho_1) + S_{2,0}e^{-q_2 T} M(d_2, d - \Sigma\sqrt{T}; -\rho_2) - Ke^{rT} M(d_1 - \sigma_1\sqrt{T}, d_2 - \sigma_2\sqrt{T}; \rho)$$

 $M(a, b; \lambda)$ is the CDF of the Standard Bivariate Normal Distribution with some correlation λ .

Monte Carlo

Apart from the Closed-Form Formula of Stulz, we mainly *estimated the fair value of a European Rainbow Option - Call on min* with the method of **Monte Carlo**.

- **Plain Monte Carlo comes along with a higher degree of computational complexity.** Therefore we cannot always take a large sample size N and for that reason we **must find more efficient ways to reduce our errors (improve accuracy)**.

$$Error \sim \frac{\sqrt{Var}}{\sqrt{N}}$$

- To achieve a better accuracy we implemented on our model two very known variance reduction techniques named : **Antithetic Variates and Control Variates**

Monte Carlo-Plain Vanilla

Given the previous equations for the Asset's Pricing Paths we can set now the following :

- $S_{i,T} = S_{i,0} \exp\{(r - \frac{1}{2}\sigma_i^2)T + \sigma_i\sqrt{T}\epsilon_i\}$ where $\epsilon_i \sim N(0, 1)$ Independent Standard Normal Random Variables.
- For each simulation $j = 0 : N$ we first generated two independent variables $U_i, i = 1, 2 \sim Unif(0, 1)$
- Then take *Inverse CDF function of Normal distribution* provided we created the standard normal variables $\epsilon_{1,j}, \epsilon_{2,j} \sim N(0, 1)$.
Thus we can accumulate all the possible prices per $j = 0 : N$ simulations and extract the payoff as it follows :
 - $Payoff_j = \max(\min(S_{1,j}, S_{2,j}) - K, 0.0)$

Monte Carlo-Plain Vanilla — cont'd

After the simulation of the payoffs per asset $i = 1, 2$ we should discount them with the continuous compounded factor e^{-rT} to obtain their present value:

- $PV = V_0 \approx e^{-rT} \frac{1}{N} \sum_{j=1}^N \text{Payoff}_{j,T}$

The above estimator converges to the true option value as $N \rightarrow \infty$ by the Law of Large Numbers w.p. = 1.

Hence, the fair estimated value of the option is the average of all payoff simulations:

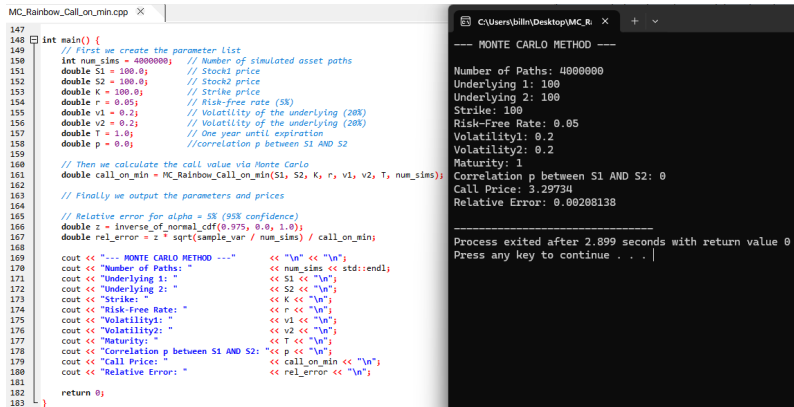
- $\hat{V}_{plain,MC} = e^{-rT} \frac{1}{N} \sum_{j=1}^N \text{Payoff}_j$

Monte Carlo Plain Vanilla — C++ Code

```
108 //Plain Monte Carlo of a Rainbow option call on the minimum
109 //Two assets with zero correlation (ρ=0)
110 double MC_Rainbow_Call_on_min(double S1, double S2, double K, double r, double v1, double v2, double T, int num_sims){
111     //Asset price at maturity --> S1,j= S0 * exp[(r-1/2 * σ1^2)T + σ1 * sqrt(T) * ε1,j]
112     //Drift terms under the risk-neutral measure:
113     double nu_T1 = (r - 0.5 * v1 * v1) * T;
114     double nu_T2 = (r - 0.5 * v2 * v2) * T;
115     //Volatility*sqrt(T)
116     double v_T1 = v1 * sqrt(T);
117     double v_T2 = v2 * sqrt(T);
118     double disc_payoff_sum = 0;
119     double disc_payoff_squared_sum = 0;
120
121     for (int i = 0; i < num_sims; i++) {
122         //Independent standard normal shocks (ρ = 0)
123         double epsilon1 = Standard_Normal_Rand();
124         double epsilon2 = Standard_Normal_Rand();
125         //Terminal prices under Black-Scholes formula:
126         double S_T1 = S1 * exp(nu_T1 + v_T1 * epsilon1);
127         double S_T2 = S2 * exp(nu_T2 + v_T2 * epsilon2);
128         //Payoff of a call on the minimum of the two assets
129         double minn = min(S_T1, S_T2);
130         double disc_payoff = max(minn - K, 0.0) * exp(-r*T);
131
132         disc_payoff_sum += disc_payoff;
133         disc_payoff_squared_sum += disc_payoff*disc_payoff;
134     }
135     //Monte Carlo price estimator
136     double disc_payoff_average = disc_payoff_sum / num_sims; // PV = (E(Payoff),T)/num_sims
137     //Unbiased sample variance of the discounted payoff
138     sample_var = (disc_payoff_squared_sum - num_sims * disc_payoff_average * disc_payoff_average) / (num_sims - 1.0);
139     return disc_payoff_average;
140 }
```

Figure: Plain(Crude) Monte Carlo - Main Function

Monte Carlo Plain Vanilla — C++ Results



The image shows a C++ program and its output. The program, `MC_Rainbow_Call_on_min.cpp`, implements a Monte Carlo simulation for a rainbow call option. It defines parameters for two underlying assets (S1 and S2), a strike price (K), risk-free rate (r), volatilities (v1, v2), time to maturity (T), and correlation (p). The simulation uses 4,000,000 paths. The output displays the calculated call price and relative error.

```
147
148 int main() {
149     // First we create the parameter list
150     int num_sims = 4000000; // Number of simulated asset paths
151     double S1 = 100.0; // Stock1 price
152     double S2 = 100.0; // Stock2 price
153     double K = 100.0; // Strike price
154     double r = 0.05; // Risk-free rate (5%)
155     double v1 = 0.2; // Volatility of the underlying (20%)
156     double v2 = 0.2; // Volatility of the underlying (20%)
157     double T = 1.0; // One year until expiration
158     double p = 0.0; // correlation p between S1 AND S2
159
160     // Then we calculate the call value via Monte Carlo
161     double call_on_min = MC_Rainbow_Call_on_min(S1, S2, K, r, v1, v2, T, num_sims);
162
163     // Finally we output the parameters and prices
164
165     // Relative error for alpha = 5% (95% confidence)
166     double z = inverse_of_normal_cdf(0.975, 0.0, 1.0);
167     double rel_error = z * sqrt(sample_var / num_sims) / call_on_min;
168
169     cout << "--- MONTE CARLO METHOD ---" << "\n" << "\n";
170     cout << "Number of Paths: " << num_sims << endl;
171     cout << "Underlying 1: " << S1 << "\n";
172     cout << "Underlying 2: " << S2 << "\n";
173     cout << "Strike: " << K << "\n";
174     cout << "Risk-Free Rate: " << r << "\n";
175     cout << "Volatility1: " << v1 << "\n";
176     cout << "Volatility2: " << v2 << "\n";
177     cout << "Maturity: " << T << "\n";
178     cout << "Correlation p between S1 AND S2: " << p << "\n";
179     cout << "Call Price: " << call_on_min << "\n";
180     cout << "Relative Error: " << rel_error << "\n";
181
182     return 0;
183 }
```

--- MONTE CARLO METHOD ---

Number of Paths: 4000000
Underlying 1: 100
Underlying 2: 100
Strike: 100
Risk-Free Rate: 0.05
Volatility1: 0.2
Volatility2: 0.2
Maturity: 1
Correlation p between S1 AND S2: 0
Call Price: 3.29734
Relative Error: 0.00208138

Process exited after 2.899 seconds with return value 0
Press any key to continue . . .

Figure: Crude Monte Carlo - Results

Monte Carlo-Antithetic Variates

Now in order to improve the efficiency and the accuracy of the Crude Monte Carlo, we introduce a variance reduction method named *Antithetic Variates*.

- This method main core is to simulate the asset paths for *both* $\epsilon_{1,j}, \epsilon_{2,j} \sim N(0, 1)$ alongside their *antithetic* -opposite sign- $-\epsilon_{1,j}, -\epsilon_{2,j}$. **With this procedure we manage to generate negatively correlated variables.**

This way we achieve the reduction of the variance of our estimator by creating symmetry, hence balancing possible high and low outcomes that would introduce noise in our results.

Monte Carlo-Antithetic Variates — cont'd

The equations used in the c++ code are provided here:

- $S_{i,j}^+(T) = S_0 \exp[(r - \frac{1}{2}\sigma_i^2)T + \sigma_i\sqrt{T}\epsilon_j]$
- $S_{i,j}^-(T) = S_0 \exp[(r - \frac{1}{2}\sigma_i^2)T + \sigma_i\sqrt{T}(-\epsilon_j)]$ where the + symbol is used for the positive ϵ_j while the - symbol represents the antithetic ϵ_j . So we can reset our payoffs in the following manner:
 - $Payoff_j^+ = \max(\min(S_{1,j}^+, S_{2,j}^+) - K, 0.0)$
 - $Payoff_j^- = \max(\min(S_{1,j}^-, S_{2,j}^-) - K, 0.0)$

Then we average those for each antithetic pair:

- $\hat{V}_j = \frac{1}{2}(Payoff_j^+ + Payoff_j^-)$ and finally the Antithetic Monte Carlo estimate for the option value is:
- $\hat{V}_{AV,MC} = e^{-rT} \frac{1}{N} \sum_{j=1}^N \hat{V}_j$ with the condition : $Var(V_{AV,MC}) < Var(V_{plain})$

Monte Carlo-Antithetic Variates — C++ Code

```
Agianoglou AV MC.cpp
107 // Pricing a European Call Option with the Monte Carlo method
108 double MC_Call_Price_Rainbow(double S1,double S2,double K,double r,double v1,double v2,double T,double p,int num_sims)
109 {
110     double nu_T1=(r-0.5*v1*v1)*T;
111     double nu_T2=(r-0.5*v2*v2)*T;
112     double v_T1=v1*sqrt(T);
113     double v_T2=v2*sqrt(T);
114
115     double anti_payoff_sum=0.0;
116     double anti_payoff_squared_sum=0.0;
117
118     for (int i=0; i<num_sims; i++){
119         double epsilon1=Standard_Normal_Rand();
120         double epsilon2=Standard_Normal_Rand();
121
122         double S_T1=S1*exp(nu_T1+v_T1*epsilon1);
123         double S_T2=S2*exp(nu_T2+v_T2*epsilon2);
124         double disc_payoff=std::max(std::min(S_T1,S_T2)-K,0.0);
125
126         //Monte Carlo with Antithetic Variates
127         double S_T1_a=S1*exp(nu_T1+v_T1*(-epsilon1));
128         double S_T2_a=S2*exp(nu_T2+v_T2*(-epsilon2));
129         double anti_payoff=std::max(std::min(S_T1_a,S_T2_a)-K,0.0);
130         double anti_payoff_squared=anti_payoff*anti_payoff;
131         double anti_mean=(disc_payoff+anti_payoff)*0.5;
132         anti_payoff_sum+=anti_mean;
133         anti_payoff_squared_sum+=(anti_mean*anti_mean);
134     }
135
136     double anti_payoff_average=anti_payoff_sum/num_sims;
137     double price_anti=(anti_payoff_average)*std::exp(-r*T);
138     anti_sample_var=((anti_payoff_squared_sum-num_sims*anti_payoff_average*anti_payoff_average)/(num_sims-1))*std::exp(-2.0*r*T);
139
140     return price_anti;
141 }
```

Figure: Monte Carlo Antithetic Variates - Main Function

Monte Carlo-Antithetic Variates — C++ Results

```
[*] Agianoglou AV MC.cpp x
143 int main() {
144     // First we create the parameter list
145     int num_sims = 4000000; // Number of simulated asset paths
146     double S1 = 100.0; // Stock1 price
147     double S2 = 100.0; // Stock2 price
148     double K = 100.0; // Strike price
149     double r = 0.05; // Risk-free rate (5%)
150     double v1 = 0.2; // Volatility of the underlying (20%)
151     double v2 = 0.2; // Volatility of the underlying (20%)
152     double T = 1.0; // One year until expiration
153     double p = 0.0; // correlation p between S1 AND S2
154
155     // Then we calculate the call value via Monte Carlo
156     double call = MC_Call_Price_Rainbow(S1, S2, K, r, v1, v2, T, p, num_sims);
157
158     // Finally we output the parameters and prices
159     // Relative error for alpha = 5% (95% confidence)
160     double z = inverse_of_normal_cdf(0.975, 0.0, 1.0);
161     double rel_error = z * std::sqrt(anti_sample_var / num_sims) / call;
162
163     cout << "---- ANTITHETIC VARIATES METHOD ----" << endl;
164     cout << "Number of Paths: " << num_sims << std::endl;
165     cout << "Underlying 1: " << S1 << "\n";
166     cout << "Underlying 2: " << S2 << "\n";
167     cout << "Strike: " << K << "\n";
168     cout << "Risk-Free Rate: " << r << "\n";
169     cout << "Volatility1: " << v1 << "\n";
170     cout << "Volatility2: " << v2 << "\n";
171     cout << "Maturity: " << T << "\n";
172     cout << "Correlation p between S1 AND S2: " << p << "\n";
173     cout << "Call Price: " << call << "\n";
174     cout << "Relative Error: " << rel_error << "\n";
175     return 0;
176 }
177
```

```
C:\Users\nikos\Downloads\Un x + v
--- ANTITHETIC VARIATES METHOD ---
Number of Paths: 4000000
Underlying 1: 100
Underlying 2: 100
Strike: 100
Risk-Free Rate: 0.05
Volatility1: 0.2
Volatility2: 0.2
Maturity: 1
Correlation p between S1 AND S2: 0
Call Price: 3.29706
Relative Error: 0.00129898

-----
Process exited after 1.608 seconds with return value 0
Press any key to continue . . .
```

Figure: Monte Carlo Antithetic Variates - Results

Monte Carlo-Antithetic Variates — cont'd

The importance of the Antithetic Variates depends in a high degree from the nature of the payoff's function. Highly (increasing) monotonic payoffs, such as rainbow options on minimum that we examine, are having strong negative correlation between the paired outcomes. This statement does not apply though for payoffs that don't exhibit this feature. Therefore the structure of each payoff is crucial when implementing the Antithetic Variates technique in Monte Carlo approximations.

Monte Carlo-Control Variates

Another way to improve accuracy and efficiency of Monte Carlo simulation is to implement the Control Variates method.

- Here the variance is again reduced by trying to introduce information to our system, from *related random variable/Control Variate*. In our case we have two new variables with the following feature :
- $E[Y_1] = \nu_1 : \text{known}$
- $E[Y_2] = \nu_2 : \text{known}$

Now we set up the controlled estimator : $X_c = X + c_1 (Y_1 - \nu_1) + c_2 (Y_2 - \nu_2)$ where the $E[X_c] = \mu$ is known. So we just have to *select the proper(optimal) c_i^** .

Monte Carlo-Control Variates —cont'd

In order to obtain those c_i^* we need to **minimize** the **Variance of the Control Variate** which is :

$$\blacksquare \text{Var}(X_c) = \text{Var}(X + c_1 (Y_1 - \nu_1) + c_2 (Y_2 - \nu_2))$$

So now we need to find the repsectively $c_{1,2}^*$ such that : $d\text{Var}(X_c)/dc_{1,2} = 0$ and then $d^2\text{Var}(X_c)/dc_{1,2}^2 > 0$.

After some algebraic operations we obtain that the optimal coefficients c_i^* , $i = 1, 2$ are:

$$\blacksquare c_1^* = -\frac{\text{Cov}(X, Y_1)}{\text{Var}(Y_1)}$$

$$\blacksquare c_2^* = -\frac{\text{Cov}(X, Y_2)}{\text{Var}(Y_2)}$$

Monte Carlo-Control Variates —cont'd

So now we can derive that from the basic control variate estimator form:

- $\hat{V}_{control} = \frac{1}{N} \sum_{j=1}^N [X^{(j)} + c_1(Y_1^{(j)} - \nu_1) + c_2(Y_2^{(j)} - \nu_2)]$
can be transformed into the *optimal variance-minimizing control variate estimator*:

- $\hat{V}_{control} = \frac{1}{N} \sum_{j=1}^N [X^{(j)} - \frac{Cov(X, Y_1)}{Var(Y_1)}(Y_1^{(j)} - \nu_1) - \frac{Cov(X, Y_2)}{Var(Y_2)}(Y_2^{(j)} - \nu_2)]$

In practice, these covariances and variances are computed from a pilot simulation.

Monte Carlo-Control Variates —cont'd

Therefore the main advantage for implying the Control Variates technique originates from the state in which the exact analytical expectation of the underlying asset is known.

Having that in mind it is straightforward that **the best control variates are the individual asset prices** : $S_1(T)$, $S_2(T)$ since their risk-neutral expectations are known analytically. So on the C++ code we setted :

- $Y_1 = S_1$
- $Y_2 = S_2$

Finally, the Control Variates method allows the user to fully adjust the estimator in a way that captures and removes parts of the variance, which occur due to any possible fluctuations between the exotic rainbow payoff and the simpler, analytical traced control variables.

Monte Carlo - Control Variates — C++ Code - Slide 1

```
116 double sample_var = 0.0;
117 // Constructing the Monte Carlo Pricing Function
118 // for Rainbow Option
119
120 double MC_Call_Price_Rainbow_CV(double S1,double S2,double K,double r,double v1,double v2,double T,int num_sims,int num_pilot){
121
122     // The correlation is assumed to be  $\rho = 0$ 
123     // Setting up the variables of the  $S1(T) = S1(0) * \exp((r-0.5 * v_1^2)T + v_1 * \epsilon_1 \sqrt{T})$ 
124
125     // Each asset here must have its own drift and volatility under the risk-neutral measure
126     double nu_1 = ( r - 0.5 * v1 * v1 ) * T;
127     double nu_2 = ( r - 0.5 * v2 * v2 ) * T;
128
129     double v_1 = v1*sqrt(T);
130     double v_2 = v2*sqrt(T);
131
132     // For the Pilot Simulations we need the : Sum of  $V1$  samples where  $V1 := S1_T$ 
133     // Same for  $V2$  samples , where  $V2 := S2_T$  as control variate
134     double S1_T_sum = 0.0;
135     double S2_T_sum = 0.0;
136
137     // Sum of  $X*V1$  from sampling in order to later estimate the  $Cov(X,V1)$ 
138     double product1_sum = 0.0;
139     // Same for the  $Cov(X,V2)$ 
140     double product2_sum = 0.0;
141
142     double disc_payoff_sum = 0.0;
143
144     // Main Loop for the path prices generation
145     int i,j;
146     for( i=0; i<num_pilot; i++){
147
148         // Generating each  $\epsilon_i \sim N(0,1)$  for asset i
149         // Here there are only 2 needed.
150
151         double epsilon1 = Stand_Normal_Rand();
152         double epsilon2 = Stand_Normal_Rand();
153
154         double S1_T = S1 * exp(nu_1 + v_1 * epsilon1);
155         double S2_T = S2 * exp(nu_2 + v_2 * epsilon2);
```

Figure: Control Variates - Main Function(1)

Monte Carlo - Control Variates — C++ Code - Slide 2

```
154 double S1_T = S1 * exp(nu_1 + v_1 * epsilon1);
155 double S2_T = S2 * exp(nu_2 + v_2 * epsilon2);
156
157 //Setting up the payoff of the 2 assets Rainbow Price
158 double minimum1 = min(S1_T, S2_T);
159
160 double disc_payoff = max(minimum1 - K, 0.0) * exp(-r * T);
161
162 // In general the formula of : Cov(X, Y_i) = E[XY_k, i] - E[X]E[Y_k, i] with k=1,2 and i : 0 up to num_pilot
163 // So the 1st term is correspondingly per asset the :
164 double product1 = S1_T * disc_payoff;
165 double product2 = S2_T * disc_payoff;
166 // Accumulate EY1_i and EY2_i
167 S1_T_sum += S1_T;
168 S2_T_sum += S2_T;
169 // Accumulate E(X_i)
170 disc_payoff_sum += disc_payoff;
171 // Accumulate E(X_i, Y1_i)
172 product1_sum += product1;
173 // Accumulate E(X_i, Y2_i)
174 product2_sum += product2;
175
176 }
177 // sample covariance estimator for Cov(X, Y) from pilot:
178 // Cov(X, Y) ≈ [ E(XY) - (E(X)E(Y))/N ] / (N-1) (PDF gives sample covariance formula for pilot)
179 double sample1_cov = (product1_sum - S1_T_sum * disc_payoff_sum / num_pilot) / (num_pilot - 1);
180 double sample2_cov = (product2_sum - S2_T_sum * disc_payoff_sum / num_pilot) / (num_pilot - 1);
181
182 // For this application the control variate is Y1 = S1_T.
183 // The PDF states E[S_T] = S0 e^{rT} and Var(S_T) = S0^2 e^{2rT} (e^{sigma^2 T} - 1).
184 double VarY1 = S1 * S1 * exp(2.0 * r * T) * (exp(v1 * v1 * T) - 1.0); // Var(Y1) known in closed form
185 double VarY2 = S2 * S2 * exp(2.0 * r * T) * (exp(v2 * v2 * T) - 1.0); // Var(Y) known in closed form
186
187
188 double ExpY_1 = S1 * exp(r * T); // E[Y1] is known
189 double ExpY_2 = S2 * exp(r * T); // E[Y2] is known
190
191 // Now we need to update the c* per asset where we obtained from minimizing Var(X_c)
192 // X_c = X + c1 * (Y1 - E[Y1]) + c2 * (Y2 - E[Y2])
193 // So : Var(X_c) = Var(X) + 2 * T * (c1 * Cov(X, Y1) + 2 * c1 * c2 * Cov(Y1, Y2))
```

Figure: Control Variates - Main Function(2)

Monte Carlo - Control Variates — C++ Code Slide 3

```
190 // Now we need to update the c* per asset where we obtained from minimizing Var(X_c)
191 // X_c = X + c1 * ( Y1 - E[Y1]) + c2 * ( Y2 - E[Y2])
192 // So : Var(X_c) = Var(X) + 2 * E ( c_i * Cov(X,Y_i) + E c_i * c_j * Cov(Yi,Yj)
193
194 // Setting up the c1* and c2*
195 double c1 = -sample1_cov/VarY1;
196 double c2 = -sample2_cov/VarY2;
197
198 double control_var_sum = 0.0;
199 double control_var_squared_sum = 0.0;
200
201 // Now we proceed to main looping iterations:
202 for(int i=0; i<num_sims; i++){
203     double epsilon1 = Stand_Normal_Rand();
204     double epsilon2 = Stand_Normal_Rand();
205
206     //Create the new S1,T,S2,T:
207     double new_S1_T = S1 * exp( nu_1 + v_1*epsilon1 );
208     double new_S2_T = S2 * exp( nu_2 + v_2*epsilon2 );
209
210     double new_minimum = min( new_S1_T, new_S2_T);
211     double new_disc_payoff = exp( -r * T ) * max( new_minimum - K, 0.0);
212
213     double control_var = new_disc_payoff + c1 * (new_S1_T - ExpY_1) + c2 * (new_S2_T - ExpY_2);
214     control_var_sum += control_var;
215     control_var_squared_sum += control_var * control_var;
216 }
217
218 double control_var_average = control_var_sum / num_sims;
219 sample_var = (control_var_squared_sum - num_sims*control_var_average*control_var_average) / (num_sims - 1);
220
221 return control_var_average;
222 }
223
224 int main() {
```

Figure: Control Variates - Main Function(3)

Monte Carlo - Control Variates — C++ Results

```
227 int main() {
228     //unsigned int seed = 123456u; // pick any fixed integer seed
229     //srand(seed);
230     // Getting the actual parameters :
231
232     int num_sims = 4000000; // Number of simulated asset paths
233     int num_pilot = 10000;
234     double S1 = 100.0; // Stock1 price
235     double S2 = 100.0; // Stock2 price
236     double K = 100.0; // Strike price
237     double r = 0.05; // Risk-free rate (5%)
238     double v1 = 0.2; // Volatility of the underlying (20%)
239     double v2 = 0.2; // Volatility of the underlying (20%)
240     double T = 1.0; // One year until expiration
241
242     // Via the function we compute the Call price of Rainbow Option
243
244     double call_price = MC_Call_Price_Rainbow_CV(S1,S2,K,r,v1,v2,T,num_sims,num_pilot);
245
246     // Relative error for alpha = 5% (95% confidence)
247     double z = inverse_of_normal_cdf(0.975, 0.0, 1.0);
248     double rel_error = z * std::sqrt(sample_var / num_sims) / call_price;
249
250     // Print out the results
251     cout << "Number of Paths for Control Variates: " << num_sims << endl;
252     cout << "Number of Pilot sims: " << num_pilot << endl;
253     cout << "Underlying 1: " << S1 << "\n";
254     cout << "Underlying 2: " << S2 << "\n";
255     cout << "Strike: " << K << "\n";
256     cout << "Risk-Free Rate: " << r << "\n";
257     cout << "Volatility1: " << v1 << "\n";
258     cout << "Volatility2: " << v2 << "\n";
259     cout << "Maturity: " << T << "\n";
260     cout << "Correlation p between S1 AND S2: 0" << "\n";
261
262     cout << "Call Price: " << call_price << "\n";
263     cout << "Relative Error: " << rel_error << "\n";
264
265     return 0;
266 }
```

```
Number of Paths for Control Variates: 4000000
Number of Pilot sims:300
Underlying 1: 100
Underlying 2: 100
Strike: 100
Risk-Free Rate: 0.05
Volatility1: 0.2
Volatility2: 0.2
Maturity: 1
Correlation p between S1 AND S2: 0
Call Price: 3.2956
Relative Error: 0.00153039
```

```
-----
Process exited after 1.547 seconds with return value 0
Press any key to continue . . .
```

Figure: Control Variates - Results

Summary of Numerical Results (Rainbow Call on min)

Method	$\hat{V}$ (Price)	Rel. Error (95%)	Runtime (s)
Crude MC	3.297 300	0.002 081	3.150 000
Antithetic Variates (AV)	3.297 000	0.001 299	2.400 000
Control Variates (CV)	3.295 600	0.001 530	1.550 000

Method	Notes / Interpretation
Crude MC	Baseline Monte Carlo; largest uncertainty for the same N .
Antithetic Variates (AV)	Best variance cancellation here due to payoff monotonicity (paired negatives).
Control Variates (CV)	Strong variance reduction and fastest runtime; min-payoff limits correlation and pilot-estimation adds some noise.

Consistency check: all estimates within $\mathcal{O}(10^{-3})$ of each other $\Rightarrow$ no evidence of bias; differences consistent with MC sampling error.

Initial parameters: $N = 4 \times 10^6$, $S_1 = S_2 = 100$, $K = 100$, $r = 0.05$, $\sigma_1 = \sigma_2 = 0.2$, $T = 1$, $\rho = 0$.